

# Task 실행 성능 기반의 Spark 애플리케이션 성능 분석

정민섭<sup>○</sup> 이동은 손민지 박정환 한환수

성균관대학교 소프트웨어대학

{hpmltsws, del96, sonmj426, j.park, hhan}@skku.edu

## Performance Analysis of Spark Application through Task Execution

Minseop Jeong<sup>○</sup>, Dongeun Lee, Minji Son, Jeonghwan Park, Hwansoo Han

College of Software, Sungkyunkwan University

### 요 약

Spark 사용자들은 목적에 따라 다른 애플리케이션을 사용하지만, 아직 목적 별 Transformation의 성능에 따른 최적의 Executor configuration 기준이 없다. 본 논문에서는 Spark RDD에서의 Transformation 성능에 따른 Executor configuration의 기준을 제시하고자 한다. Spark Executor configuration과 인풋 파일 사이즈에 따른 Transformation 성능 측정을 하는 실험을 진행하여 Transformation 별 전체 Job 수행 시간에 영향을 미치는 정도를 파악하였다. 또한 Transformation의 입력 데이터 크기와 configuration에 따른 상대적 성능을 비교하였고 이에 대해 분석하였다.

### 1. 서 론

최근 급격하게 증가한 SNS, 기업들의 고객 정보 수집, 미디어 콘텐츠와 사용에 대한 정보들로 인해 단일 노드에서의 데이터 처리 방식으로는 대용량의 정보를 처리하기 힘들어졌다. 따라서 빅데이터를 처리하는 기법에 대한 수요와 관심이 증가하고 있다. 현재 가장 많이 쓰이고 있는 빅데이터 분석 프레임워크로 Spark, Hadoop 등이 있다.

Spark에서는 여러 분산 노드에 걸쳐서 RDD라는 추상 데이터 구조에, 처리가 필요한 실제 데이터를 저장한다. RDD는 읽기 전용이며 Transformation 작업을 통해 하나의 RDD로부터 새로운 RDD를 생성하는 것이 가능하다. 각 노드에는 작업을 수행하는 Executor들이 존재하는데, 모든 Executor들은 그에 따른 configuration을 바탕으로 구성되고 각 RDD를 Transformation 작업을 반복하면서 원하는 정보 처리 작업을 수행한다. 이러한 Transformation의 반복은 RDD Action 연산을 통해 사용자가 알아볼 수 있는 형태의 데이터로 가공하는 것으로 완료한다.

Spark 사용자들은 데이터 분석에 필요한 애플리케이션을 목적에 따라 다르게 작성하여 사용한다. 사용 목적에 따라 data-intensive한 작업을 필요로 하거나 compute-intensive한 작업을 필요로 한다. 이러한 용도에 따라 생성 되는 RDD의 형태와 처리하는 방법도 달라진다. 따라서 Executor의 configuration에 따라 Spark의 작업 성능이 달라질 텐데, 현재 애플리케이션의 종류에 따른 최적의 Executor configuration에 대한 가이드라인은 제공되지 않는다.

본 논문에서는 Spark Transformation 성능에 따른 configuration의 기준을 제시하고자 한다. Spark의 각 RDD 연산의 종류와 입력 데이터 크기에 따른 Task 성능 측정 실험을 통해 그 결과를 분석하였다. 실험 결과를 보면 입력 데이터의 크기가 메모리의 총량을 넘지 않을 때는 core의 개수가 많을수록 성능이 좋은 경향을 보였다. 입력 데이터의 크기가 메모리의 총량보다 커지면 core의 개수가 많을수록 오히려 성능이 나빠지는 경향을 보였다. 또한 총 작업시간의 패턴에 대한 분석을 진행하였다.

### 2. 배경 지식

#### 2.1 Spark

Spark는 사용자가 대용량 시스템에서 대규모 데이터 분석 응용 프로그램을 실행할 수 있게 해주는 오픈 소스 프레임워크이다. Spark는 Hadoop과 같은 여타 빅데이터 분석 프레임워크와 비교하여 처리 속도가 빠르다는 장점이 있다. Spark는 메모리에 데이터를 상주시키면서 처리하기에 다른 프레임워크에 비해 디스크 I/O 비율이 낮기 때문이다.

#### 2.2 RDD

RDD는 Spark 내에서 여러 분산 노드에 걸쳐서 저장되는 데이터의 집합이다. Spark에서의 작업은 새로운 RDD를 만들거나 기존에 존재하는 RDD를 변형하는 것, RDD에서 연산을 호출하는 것 중의 하나로 표현된다. RDD의 중요한 특징 중 하나는 읽기 전용 데이터인 것으로 RDD가 한번 생성 되면 그 이후로 업데이트되지 않기 때문에 데이터 일관성에 관한 문제가 발생하지 않는다.

#### 2.3 Spark execution model

Spark는 하나의 RDD를 처리하기 위해 한 번에 한 개의 Task가 하나의 Executor에 할당되어 데이터를 처리한다. 이때, 각 Task가 처리하는 데이터의 기본 단위는 128MB로 구성된 하나의 Block으로, 이는 Hadoop 분산 파일시스템의 기본 입출력 단위이다. 이로 인해 기존의 실행 모델에서는 하나의 Executor에 대하여 할당된 자원의 양과 관계없이 항상 일정한 양의 데이터를 처리해야한다.

### 3. 관련 연구

Spark는 애플리케이션의 종류와 워크로드, 입력 데이터의 크기 등에 따라 각각 다른 configuration에서 최적의 성능을 보인다. 본 장에서는 HPSSD, SparkBench, Hibenbch를 사용해 최적의 성능으로 Spark를 활용할 수 있는 configuration에 대한 관련연구를 다룬다.

#### 3.1 Spark with High Performance SSD

보통 Spark 클러스터에서 slave 노드의 CPU 코어의 수가 늘어나면 성능이 증가한다. 하지만, 코어의 수 외에 메모리 configuration도 Spark의 성능에 큰 영향을 준다. PageRank

본 연구는 2017년도 미래창조과학부의 재원으로 한국연구재단의 지원을 받아 PF급 이종 초고성능컴퓨터 개발(NRF-2016M3C4A7952587)의 연구결과와 과학기술정보통신부 및 정보통신기술진흥센터의 SW중심대학지원사업(2015-0-00914)의 연구결과로 수행되었음

표 1 실험 환경

|                 |                           |
|-----------------|---------------------------|
| Spark Version   | 2.1.0                     |
| Hadoop Version  | 2.7.2                     |
| Number of cores | 24                        |
| Memory size     | 64GB                      |
| Storage         | Samsung SSD 850 Pro 512GB |

표 2 Executor configuration 별 자원 할당 현황

|         | memory/Executor | Executor 개수 |
|---------|-----------------|-------------|
| Config1 | 2GB             | 24개         |
| Config2 | 3GB             | 16개         |
| Config3 | 4GB             | 12개         |

워크로드를 Spark 클러스터에서 실행하는 동안 각 데이터 노드의 코어 개수를 2~12개로 배치하여 실험한다. 그 결과 코어의 수를 늘릴수록 이상적인 메모리 대역폭과 실제로 하나의 slave 노드에서 사용하는 메모리 대역폭과의 차이가 커지며, 이는 Spark의 성능에도 영향을 미친다. 12 코어의 configuration을 사용하는 경우 이상적인 경우의 60%의 메모리 대역폭 사용량을 보이며, Spark 성능도 이상적인 경우의 60%로 나타난다. 즉, CPU 코어의 수를 추가할수록 메모리 대역폭 사용량의 증가율이 감소하여 전체 Spark의 성능을 제한한다. 때문에 높은 Spark 성능을 위해서는 적당한 CPU와 메모리 configuration이 필요하다.

### 3.2 SparkBench

SparkBench는 다양한 어플리케이션의 집합으로 Spark의 개발에 더 나은 자원 configuration을 정량적으로 비교할 수 있도록 한다. 각각의 가상 머신은 하나의 Executor를 갖고, 각 Executor마다 가상 CPU 코어의 수를 1~8개로 배치하여 실험한다. Logistic regression, PageRank, Hive의 3가지 워크로드에 대해 실험한 결과, 코어가 4개인 경우까지는 총 실행 시간이 줄어들지만, 코어가 6개일 때는 4개일 때와 거의 차이가 없으며, 8개의 경우에는 오히려 실행시간이 증가하게 된다. 즉, CPU 자원을 충분히 활용하기 위해 Task 병렬성을 높이면 실행시간이 줄어들지만, 너무 과도하게 높이면 CPU contention이 발생해 실행시간에 악영향을 끼치게 된다. CPU contention을 줄이면서도 CPU 자원을 최대한 활용할 수 있는 CPU configuration을 구하는 것이 중요하다.

### 3.3 Hibench

Hibench는 Intel이 개발한 다양한 어플리케이션의 세트로 컴퓨터 시스템의 성능을 측정하는데 실험적인 접근법을 제시한다. Pagerank, Sort, Terasort, Aggregation, Scan의 5가지 Hibench 워크로드에 대해 실험한 결과, Spark가 Apache Hadoop보다 평균적으로 11% 정도 높은 메모리 사용률을 보인다. 즉, Spark의 Executor는 연산에 필요한 데이터를 메모리에 저장하고 필요한 정보를 캐싱하기 때문에 Apache Hadoop보다 더 많은 메모리 자원을 소비하기 때문에 사용할 수 있는 메모리가 적은 경우 새로운 RDD를 만들 공간이 부족해 전체 성능이 제한된다. 워크로드에 따라 프로세스를 저장하거나 새로운 RDD를 만들어야 하는 공간의 양이 달라지기 때문에 메모리 사용률을 고려하여 최적의 메모리 configuration을 구해야 한다.

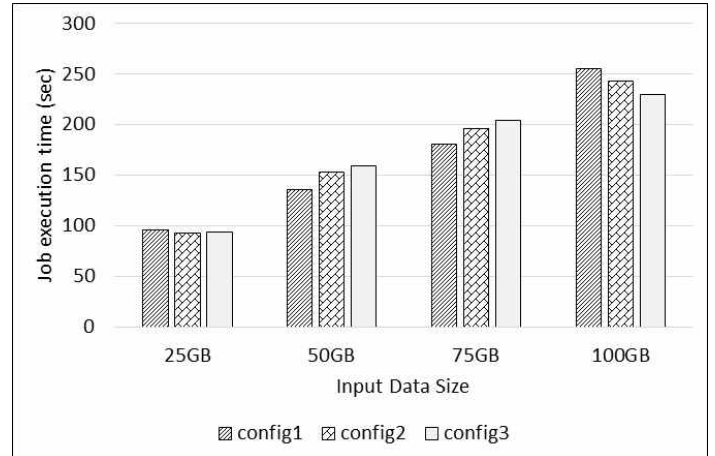


그림 3 입력 데이터 크기 및 configuration 별 Job의 총 수행 시간 비교 그래프

## 4. 실험 결과

### 4.1 실험 환경

Spark Executor configuration과 인풋 파일 사이즈 별 RDD transformation의 성능을 분석하기 위해 표 1과 같은 환경을 갖춘 노드 3대에서 성능을 측정하였다. 이 중 1대의 노드를 Master 노드로, 2대의 노드를 Slave 노드로 사용하였다.

하나의 Executor에 대한 configuration은 표 2와 같이 구성하였다. 이는 각 노드에서 사용하는 총 메모리 사용량을 48GB로 통일하도록 구성된 수치이다. 그 이유는 같은 메모리 용량을 가진 시스템에서 core 수를 제한하였을 경우 어떤 성능 패턴을 보이는지 분석하기 위함이다. 하지만 48GB의 메모리 전체를 연산에만 사용할 수는 없는데, 그 이유는 이 중 40% 가량을 캐시에 사용하기 때문이다. 이를 감안하여 노드 1대당 약 30GB 가량의 메모리를 연산에 사용하도록 설계하였다. 이와 함께 데이터 크기별로 나타나는 성능 패턴을 파악하기 위해 하나의 파일로 구성된 25GB, 50GB, 75GB, 100GB의 4가지 입력 데이터를 사용하였다.

### 4.2 실험 결과 및 분석

그림 1은 configuration 별 데이터 크기에 따른 Job의 총 수행 시간을 비교한 그래프이다. 해당 그래프에 표기된 Config1, 2, 3은 표 2에 제시한 configuration이다. 이 때, 25GB의 실험에서 각 configuration 별 성능 차이는 4% 미만으로 오차 범위 내의 성능을 보였다. 그리고 입력 데이터의 크기가 Spark 환경에서의 연산 메모리의 총량을 넘지 않는

표 3 RDD 식별자 정의 및 성능 영향 척도

| RDD# | Name                    | Task Impact Factor |
|------|-------------------------|--------------------|
| 1    | CartesianRDD            | 2.8                |
| 2    | CoalescedRDD            | 2.7                |
| 3    | CoGroupedRDD            | 20.1               |
| 4    | PartitionwiseSampledRDD | 3.6                |
| 5    | PipedRDD                | 6.1                |
| 6    | ShuffledRDD_large       | 36.5               |
| 7    | ShuffledRDD_medium      | 17.1               |
| 8    | ShuffledRDD_small       | 8.9                |
| 9    | UnionRDD                | 2.1                |

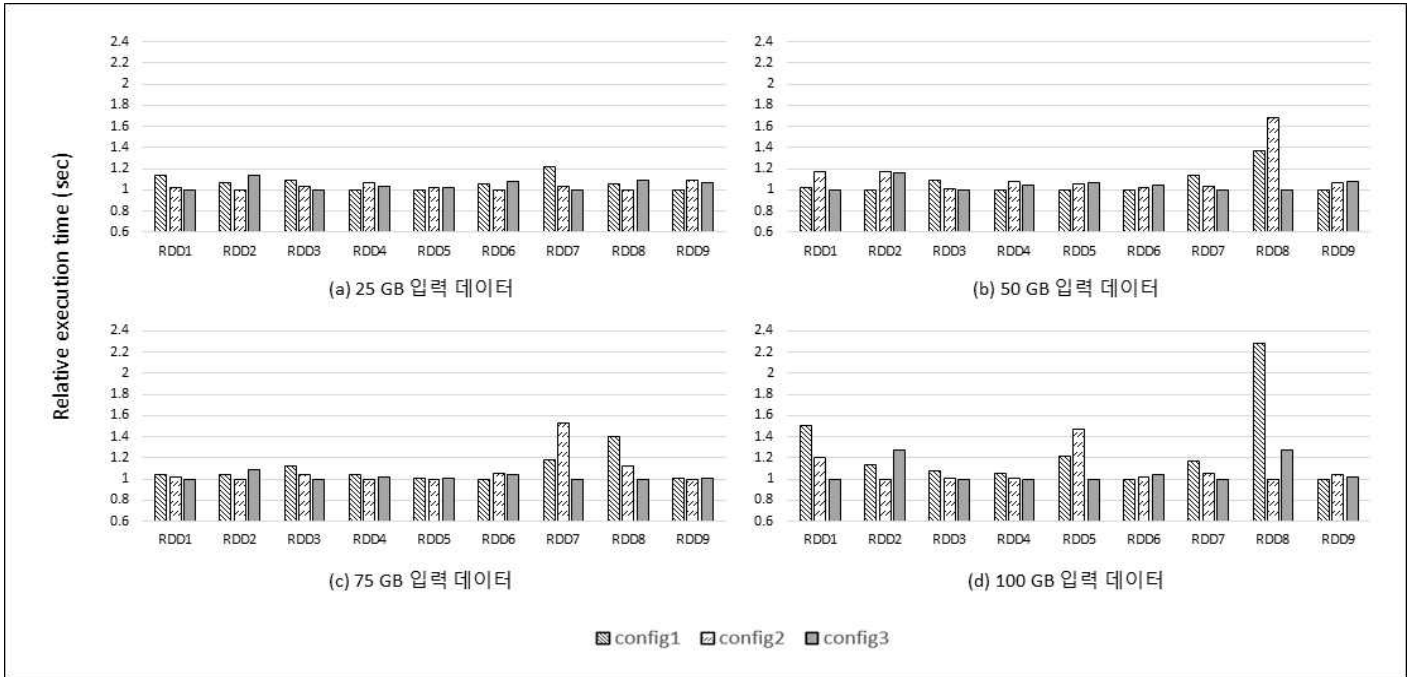


그림 4 데이터 크기 및 configuration에 따른 RDD 상대 성능 그래프

50GB와 75GB의 실험에서는 core의 개수가 많을수록 성능이 좋은 경향을 보였다. 하지만 메모리 총량을 넘는 데이터를 입력받는 100GB에서는 core의 개수를 많이 사용하는 configuration에서 오히려 성능이 나빠지는 성능 역전 현상이 나타난다. 이러한 현상이 나타나는 가장 큰 이유는 자바 가상 머신(Java Virtual Machine)의 쓰레기 수집(Garbage collection) 작업 때문이다. Spark의 각 Task는 하나의 Executor가 얼마만큼의 메모리를 할당받는지와 관계없이 일정한 양의 데이터를 처리해야하기 때문에, Executor 당 메모리 할당량이 적은 configuration에서 쓰레기 수집 작업을 더 오랜 시간동안 수행하게 된다.

표 3은 각 그림에 표기된 RDD1-9까지의 종류 및 총 실행 시간 대비 각 RDD 연산의 수행 시간을 나타낸 표이다. 이 중, 각 RDD 연산에 대해 하나의 Task가 전체 수행 시간에 영향을 미치는 정도를 Task Impact Factor(TIF)로 정의하였다. TIF는 전체 수행 시간 대비 하나의 RDD를 처리하는데 걸리는 시간으로 정하였다. 또한 RDD 항목 중 RDD6-8에 해당하는 ShuffledRDD들은 TIF가 중간 입력 데이터 크기에 따라 큰 차이를 보이기 때문에 3가지로 분류하였다.

그림 2는 앞서 설명한 총 작업시간의 패턴을 미시적으로 설명하기 위한 그래프이다. 우선 그림 2-(a)의 경우, configuration 별로 큰 성능차이를 보이는 RDD는 20%의 최대 성능 차이를 보이는 ShuffledRDD\_medium이다. 이 때, 해당 RDD의 TIF는 비교적 크기 때문에 전체적인 성능에 영향을 미친 것이라 파악할 수 있다. 그림 2-(b)의 경우는 ShuffledRDD\_small이 매우 높은 수치로 성능 차이를 보였으나, 이는 TIF가 비교적 낮은 RDD이기 때문에 전체 성능을 뒤집기에는 어려움이 있다. 그림 2-(c)에서는 3번째로 높은 TIF 수치를 가지는 ShuffledRDD\_medium이 50%가 넘는 차이를 보인다. 하지만 Job 전체의 수행 시간은 config2가 2번째의 실행 성능을 보였다. 이는 입력 데이터가 Executor의 총 연산 메모리보다 큰 경우에 대한 실험으로, 성능에 영향을 미치는 다른 변수가 추가된 상황이다. 이러한 이유로 Task의 성능 패턴이 Job의 실행 시간과는 다른 양상을 보인 것으로 분석된다. 마지막으로 그림 2-(d)는 하나의 Executor가 처리해야할 데이터의 양이 매우 많은 경우이다. 이로 인해 core 수가 많아질수록 Executor에 로드되는 데이터의 양이 증가하고, 결국 스토리지의 대역폭을

초과하는 경우가 발생한다. 이로 인해 많은 양의 메모리 자원을 가진 Executor가 오히려 많은 RDD 연산에서 낮은 성능을 보이는 것을 알 수 있다.

## 5. 결론 및 향후 연구

본 논문에서는 Spark의 Executor 수준에서 RDD 연산의 성능을 측정하고, 이를 바탕으로 전체 Job의 성능 패턴을 분석하였다. 분석 결과, 몇몇의 RDD 연산이 전체 Job 성능에 영향을 미치는 추세를 보였으며 이를 통해 전체적인 성능 향상을 위해 각 애플리케이션의 목적에 따라 필요한 성능 가이드라인을 제시하였다.

하지만 실험 결과 중 Executor 수준의 RDD 연산만으로는 설명할 수 없는 패턴이 존재하는데, 본 연구에서는 이를 설명할 다른 요인을 찾지 못했다. 이를 위해서 추가적인 TIF를 찾고, 최종적으로 전체 성능에 대한 추론을 통해 동적으로 Executor의 configuration을 변환하여 최적의 성능을 발휘할 수 있는 프레임워크를 추후에 연구하고자 한다.

## 참 고 문 헌

- [1] Spark.apache.org, "Spark Configuration - apache spark," 2016. Available: <https://spark.apache.org>
- [2] Stephen Choi, Weiqing Yang, Yang-Suk Kee, "Early Experience with Optimizing I/O Performance Using High-Performance SSDs for In-Memory Cluster Computing," *IEEE International Conference on Big Data*, pp. 1073-1083, 2015.
- [3] Min Li, Jian Tan, Yandong Wang, Li Zhang, Valentina Salapura, "SPARKBENCH: A Comprehensive Benchmarking Suite For In Memory Data Analytic Platform Spark," *Proc of the 12th ACM International Conference on Computing Frontiers*, pp. 53, 2015.
- [4] Yassir Samadi, Mostapha Zbakh, Claude Tadonki, "Comparative study between Hadoop and Spark based on Hibenbch benchmarks," *IEEE Internation Conference on Cloud Computing Technologies and Applications*, pp. 267-275, 2016.